

Exercise 3

More list manipulation

We'll start off doing a bit more list manipulation, looking at some list comprehensions and pattern matching.

List comprehensions & pattern matching

For this section, the template code is [lists-exercise3.hs](#).

Let's first look at some pattern matching, and combination with guard expressions.

Exercise 3.1

Write a function `compress :: Eq a => [a] -> [a]` that eliminates consecutive duplicate elements of a list, but otherwise leaves the order unchanged.

Now some simple list comprehensions. A [pythagorean triple](#) is a tuple (x,y,z) of positive integers where $x^2+y^2=z^2$. For our first go, we won't care about ordering, so we'll allow generation of both $(3,4,5)$ and $(4,3,5)$ (for example).

Exercise 3.2

Using a list comprehension, define `pyths :: Int -> [(Int, Int, Int)]` that generates all pythagorean triples with components less than or equal to the specified integer.

For example

```
Prelude> pyths 5
[(3, 4, 5), (4, 3, 5)]
```

Note that integer exponentiation in Haskell is written `x^y`.

Exercise 3.3

Now modify your answer to only produce unique triples (don't worry about ordering), so for example

```
Prelude> pyths' 5
[(3, 4, 5)]
```

Recall that when introducing variables in a list comprehension, later generators can refer to the variables introduced by earlier ones.

Question 3.4

The power two is special here, indeed [Fermat's Last Theorem](#), proved by Andrew Wiles in 1994, states that the equation $x^n + y^n = z^n$ has no solutions for positive integers x, y, z, n when $n > 2$.

You could try and confirm this for $n=3$ by checking that the list generated by a comprehension with $x^3 + y^3 = z^3$ is empty.

Would you be able to use this to prove the theorem? If not, why not?

More than one way to do it

Unlike Python for which the [zen of python](#) says that

“There should be one – and preferably only one – obvious way to do it.”

In Haskell there are often multiple different ways to do the same thing, each of which may be more or less obvious depending on what you're used to.

Let's look at this by defining a function to compute the scalar product of two vectors (represented as lists) of numbers of length n . Recall that the scalar (or dot) [product](#) is defined as

$$a \cdot b = \sum_{i=1}^N a_i b_i$$

Exercise 3.5

Define a function to compute the scalar product in three different ways

1. Using `sum` and a list comprehension
2. Using `sum`, `map`, and `zip`.
3. Using `sum` and `zipWith`

The template code has a few more explanatory comments.

You may assume that both vectors have the same length.

Question 3.6

Which one do you prefer, and why?